ФКН

Master's Programme
«Data Science»

Research project for
«Ordered Sets in Data Analysis»

Predicting Telco Customer Churn with Lazy formal concept analysis

Students:
Raimova Alina
Abramov Aleksandr

Higher School of Economics
Faculty of Computer science

Dataset

Telco Customer Churn

https://www.kaggle.com/datasets/blastchar/telco-customer-churn

Objects: 7032
Train set: 6328
Test set: 704

Numerical features

- tenure
- MonthlyCharges
- TotalCharges

Target: Churn

Yes, if the customer left.
No, if the customer stayed with the company.

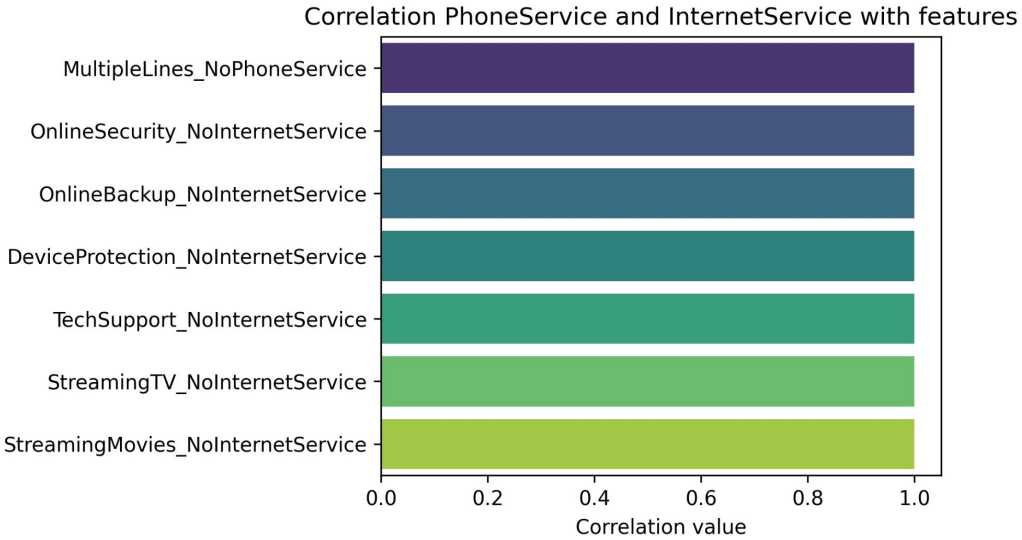
Categorical features

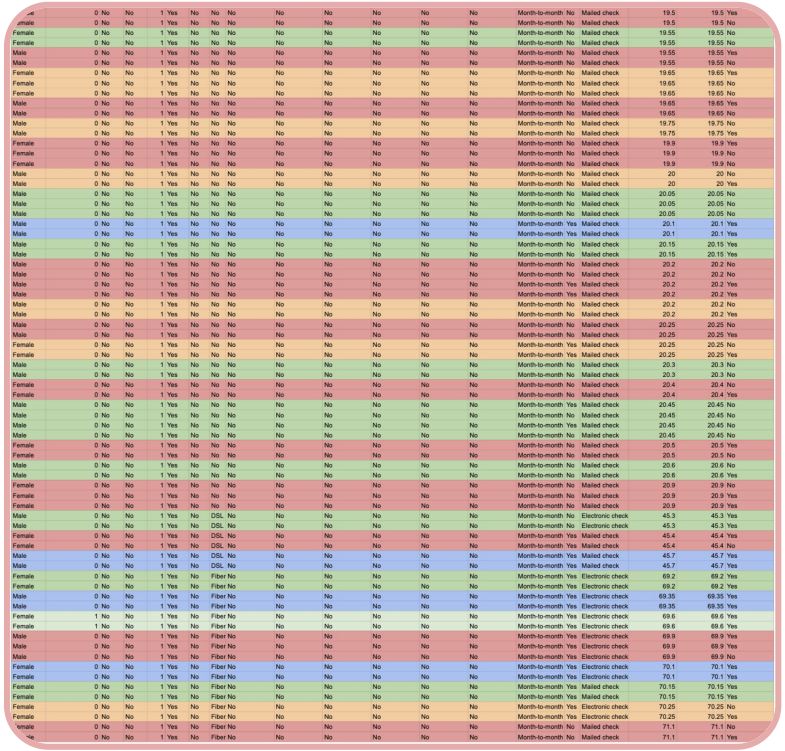
- gender
- Partner
- Dependents
- PhoneService
- MultipleLines
- InternetService
- OnlineSecurity
- OnlineBackup
- DeviceProtection
- TechSupport
- StreamingTV
- StreamingMovies
- Contract
- PaperlessBilling
- PaymentMethod

Exploratory data analysis

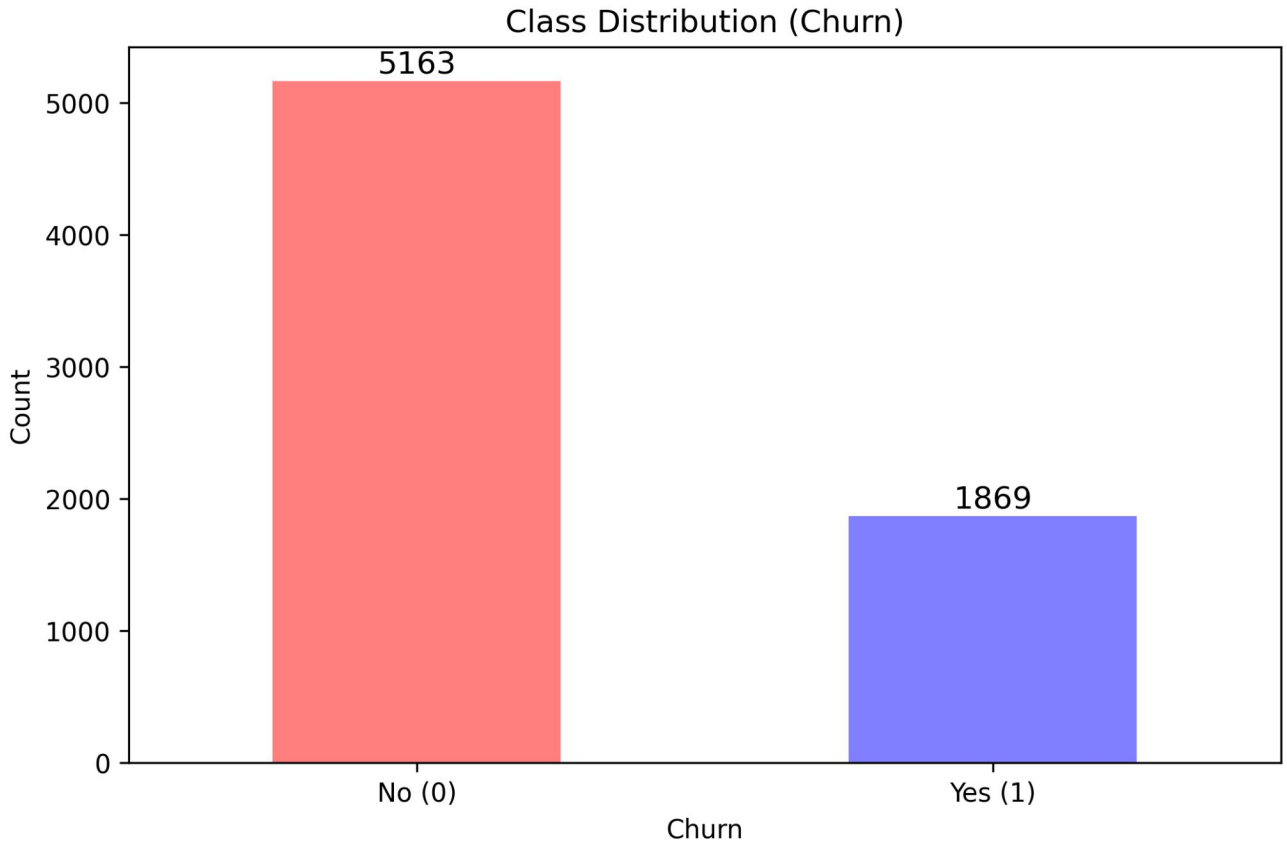
- 11 rows have missing values in the dataset – empty string for TotalCharges

```
print(f"Number of empy (space) string in cell: {df[df['TotalCharges'] == ' '].shape[0]}")
✓ 0.0s
Number of empy (space) string in cell: 11
```





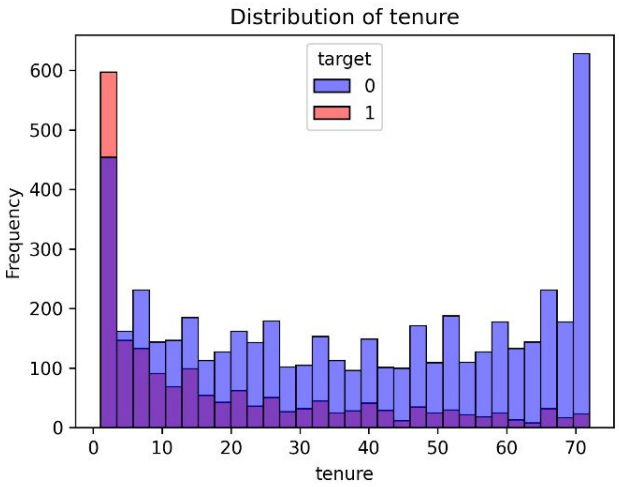
EDA, target variable

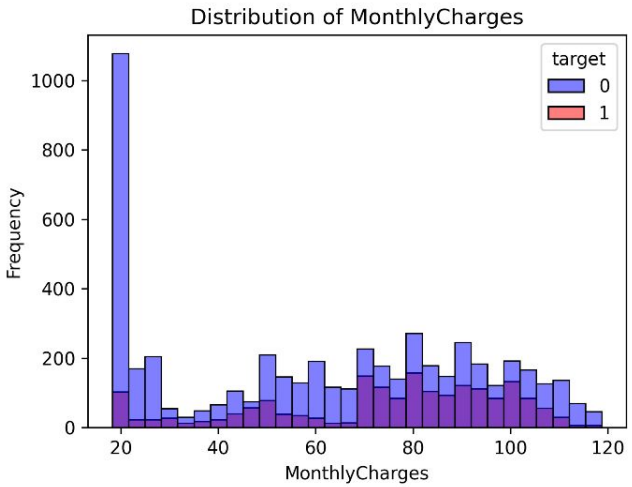


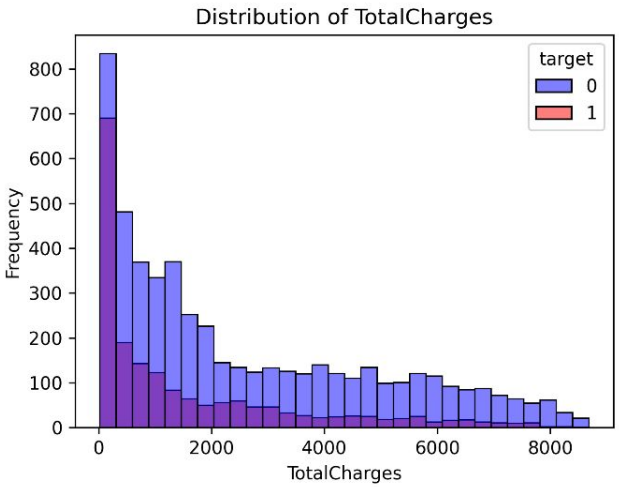
Hugely unbalanced classes

Empirical threshold for defining the negative class is 5163 / 1869 ≈ 2.76

EDA, numerical features

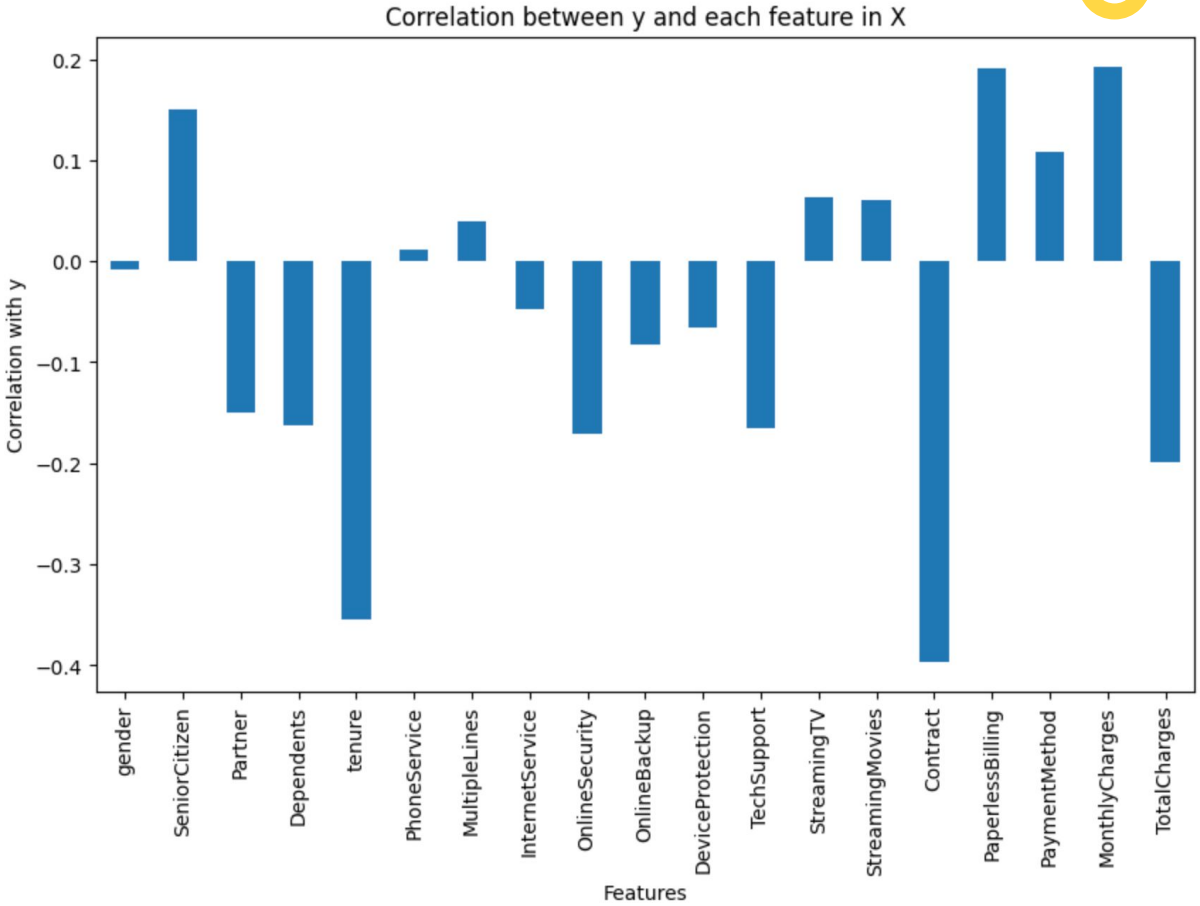






- Values 0 – 20 for most objects in the positive class
- Values 10 – 40 for most negative objects
- No noticeable difference in distribution after 40
- Does not highlight significant patterns for classification

EDA, correlations with target



«Vanilla» interval pattern structures


```python
class LazyClassifierFCA:
    def fit(self, X_train: pd.DataFrame, y_train: pd.Series) -> None:
        self.X_train = X_train
        self.y_train = y_train

    def classify_sample(self, sample: pd.Series) -> Any:
        X_train_positive = self.X_train[y_train == 1]
        X_train_negative = self.X_train[y_train == 0]

        binary_X_train_positive = X_train_positive.select_dtypes(include='bool')
        numerical_X_train_positive = X_train_positive.select_dtypes(exclude='bool')
        binary_X_train_negative = X_train_negative.select_dtypes(include='bool')
        numerical_X_train_negative = X_train_negative.select_dtypes(exclude='bool')

        def find_number_of_matches(binary_intersection, numerical_intervals):
            binary_positive_match = (binary_X_train_positive | ~binary_intersection).all(axis=1)
            binary_negative_match = (binary_X_train_negative | ~binary_intersection).all(axis=1)
            is_match = lambda row: all(interval[0] <= value <= interval[1] for value, interval in zip(row, numerical_intervals))
            numerical_positive_match = numerical_X_train_positive.apply(is_match, axis=1)
            numerical_negative_match = numerical_X_train_negative.apply(is_match, axis=1)
            return (binary_positive_match & numerical_positive_match).sum(),  (binary_negative_match & numerical_negative_match).sum()
        def is_positive_classifier(binary_intersection, numerical_intervals):
            num_positive, num_negative = find_number_of_matches(binary_intersection, numerical_intervals)
            return num_negative == 0 and num_positive > 1
        def is_negative_classifier(binary_intersection, numerical_intervals):
            num_positive, num_negative = find_number_of_matches(binary_intersection, numerical_intervals)
            return num_positive == 0 and num_negative > 1

        positive_classifiers, negative_classifiers = 0, 0
        for _, pos_sample in X_train_positive.iterrows():
            binary_intersection = sample[binary_X_train_positive.columns] & pos_sample[binary_X_train_positive.columns]
            numerical_intersection = [(min(sample[col], pos_sample[col]), max(sample[col], pos_sample[col])) for col in numerical_X_train_positive.columns]
            if is_positive_classifier(binary_intersection, numerical_intersection): positive_classifiers += 1
        for _, neg_sample in X_train_negative.iterrows():
            binary_intersection = sample[binary_X_train_negative.columns] & neg_sample[binary_X_train_negative.columns]
            numerical_intersection = [(min(sample[col], neg_sample[col]), max(sample[col], neg_sample[col])) for col in numerical_X_train_negative.columns]
            if is_negative_classifier(binary_intersection, numerical_intersection): negative_classifiers += 1
        if positive_classifiers > negative_classifiers: return 1
        elif negative_classifiers > positive_classifiers: return 0
        else: return 1

    def predict(self, X_test: pd.DataFrame) -> List[Any]:
        return [self.classify_sample(sample) for _, sample in tqdm.tqdm(X_test.iterrows(), total = len(X_test))]
```

```python
classifier = LazyClassifierFCA()
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
```

[8]

```
0%|          | 2/704 [04:08<24:15:26, 124.40s/it]
```

- Replace slow pandas with fast numpy wherever possible
- Use joblib to compute predictions for different objects in parallel
- Do as many calculations during fit as possible
- Vectorize the computations and remove for-loops, especially nested
- Cache intermediate results

IPS optimization, part 1


```python
def fit(self, X_train: pd.DataFrame, y_train: pd.Series) -> None:
    self.X_train = X_train
    self.y_train = y_train

def classify_sample(self, sample: pd.Series) -> Any:
    # Split X_train into positive and negative classes
    X_train_positive = self.X_train[y_train == 1]
    X_train_negative = self.X_train[y_train == 0]

    positive_classifiers = 0
    negative_classifiers = 0

    # Split the each part into binary and numerical
    binary_X_train_positive = X_train_positive.select_dtypes(include='bool')
    numerical_X_train_positive = X_train_positive.select_dtypes(exclude='bool')
    binary_X_train_negative = X_train_negative.select_dtypes(include='bool')
    numerical_X_train_negative = X_train_negative.select_dtypes(exclude='bool')
```

```python
class LazyClassifierFCA:
    def fit(self, X_train: pandas.DataFrame, y_train: pandas.Series):
        self.bool_columns = list(X_train.columns[X_train.dtypes == 'bool'])
        self.numeric_columns = list(X_train.columns[X_train.dtypes != 'bool'])

        self.bin_X_train_pos = (
            X_train[y_train == 1][self.bool_columns].to_numpy().astype(bool)
        )
        self.num_X_train_pos = (
            X_train[y_train == 1][self.numeric_columns].to_numpy()
        )
        self.bin_X_train_neg = (
            X_train[y_train == 0][self.bool_columns].to_numpy().astype(bool)
        )
        self.num_X_train_neg = (
            X_train[y_train == 0][self.numeric_columns].to_numpy()
        )
```

```python
def predict(self, X_test: pd.DataFrame) -> List[Any]:
    # List to store predictions for each test sample
    predictions = []

    # Iterate through each sample in X_test
    for _, sample in X_test.iterrows():
        # Classify the sample and append the result to predictions
        prediction = self.classify_sample(sample)
        predictions.append(prediction)

    return predictions
```

```python
def predict(self, X_test: pandas.DataFrame):
    return joblib.Parallel(n_jobs = -1)(
        joblib.delayed(self.classify_sample)(sample)
        for _, sample in tqdm.tqdm(
            X_test.iterrows(),
            total = len(X_test)
        )
    )
```

IPS optimization, part 2


```python
def find_number_of_positive_and_negative_matches(binary_intersection, numerical_intervals):
    # Binary condition: Matches the binary intersection
    binary_positive_match = (binary_X_train_positive | ~binary_intersection).all(axis=1)
    binary_negative_match = (binary_X_train_negative | ~binary_intersection).all(axis=1)

    # Numerical condition: Lies within the intervals
    numerical_positive_match = numerical_X_train_positive.apply(
        lambda row: all(
            interval[0] <= value <= interval[1] for value, interval in zip(row, numerical_intervals)
        ),
        axis=1
    )
    numerical_negative_match = numerical_X_train_negative.apply(
        lambda row: all(
            interval[0] <= value <= interval[1] for value, interval in zip(row, numerical_intervals)
        ),
        axis=1
    )
    # Combine binary and numerical conditions
    num_positive = (binary_positive_match & numerical_positive_match).sum()
    num_negative = (binary_negative_match & numerical_negative_match).sum()
    return num_positive, num_negative
```

```python
def is_positive_classifier(binary_intersection, numerical_intervals):
    num_positive, num_negative = find_number_of_positive_and_negative_matches(binary_intersection, numerical_intervals)
    return num_negative == 0 and num_positive > 1

def is_negative_classifier(binary_intersection, numerical_intervals):
    num_positive, num_negative = find_number_of_positive_and_negative_matches(binary_intersection, numerical_intervals)
    return num_positive == 0 and num_negative > 1

for _, pos_sample in X_train_positive.iterrows():
    binary_intersection = sample[binary_X_train_positive.columns] & pos_sample[binary_X_train_positive.columns]
    numerical_intersection = [
        (min(sample[col], pos_sample[col]), max(sample[col], pos_sample[col]))
        for col in numerical_X_train_positive.columns
    ]
    if is_positive_classifier(binary_intersection, numerical_intersection):
        positive_classifiers += 1

for _, neg_sample in X_train_negative.iterrows():
    binary_intersection = sample[binary_X_train_negative.columns] & neg_sample[binary_X_train_negative.columns]
    numerical_intersection = [
        (min(sample[col], neg_sample[col]), max(sample[col], neg_sample[col]))
        for col in numerical_X_train_negative.columns
    ]
    if is_negative_classifier(binary_intersection, numerical_intersection):
        negative_classifiers += 1
```

```python
def get_num_matches(
    self,
    bin_X_train: numpy.ndarray, num_X_train: numpy.ndarray,
    bin_int: numpy.ndarray, num_int_min: numpy.ndarray, num_int_max: numpy.ndarray
):
    bin_match = (bin_X_train | ~bin_int).all(axis = 1)
    num_match = ((num_int_min <= num_X_train) & (num_X_train <= num_int_max)).all(axis = 1)
    return (bin_match & num_match).sum()

def find_matches_impl(self, bin_int: numpy.ndarray, num_int_min: numpy.ndarray, num_int_max: numpy.ndarray):
    num_positive = self.get_num_matches(
        self.bin_X_train_pos, self.num_X_train_pos,
        bin_int, num_int_min, num_int_max
    )
    num_negative = self.get_num_matches(
        self.bin_X_train_neg, self.num_X_train_neg,
        bin_int, num_int_min, num_int_max
    )
    return num_positive, num_negative
```

```python
def find_matches(self, sample_bin: numpy.ndarray, sample_num: numpy.ndarray, bin: numpy.ndarray, num: numpy.ndarray):
    num_stacked = numpy.vstack([ sample_num, num ])
    return self.find_matches_impl(sample_bin & bin, num_stacked.min(axis = 0), num_stacked.max(axis = 0))

def get_number_of_classifiers(self, sample_bin, sample_num, bin_X_train, num_X_train, is_classifier):
    num_classifiers = 0
    for bin, num in zip(bin_X_train, num_X_train):
        if is_classifier(*self.find_matches(sample_bin, sample_num, bin, num)):
            num_classifiers += 1
    return num_classifiers

def classify_sample(self, sample: pandas.Series):
    def is_positive_classifier(num_positive, num_negative):
        return num_negative == 0 and num_positive > 1
    def is_negative_classifier(num_positive, num_negative):
        return num_positive == 0 and num_negative > 1

    sample_bin = sample[self.bool_columns].to_numpy().astype(bool)
    sample_num = sample[self.numeric_columns].to_numpy()
    positive_classifiers = self.get_number_of_classifiers(
        sample_bin, sample_num, self.bin_X_train_pos, self.num_X_train_pos, is_positive_classifier
    )
    negative_classifiers = self.get_number_of_classifiers(
        sample_bin, sample_num, self.bin_X_train_neg, self.num_X_train_neg, is_negative_classifier
    )
```

Optimized «vanilla» IPS


```
classifier = LazyClassifierFCA()
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
```

[8]

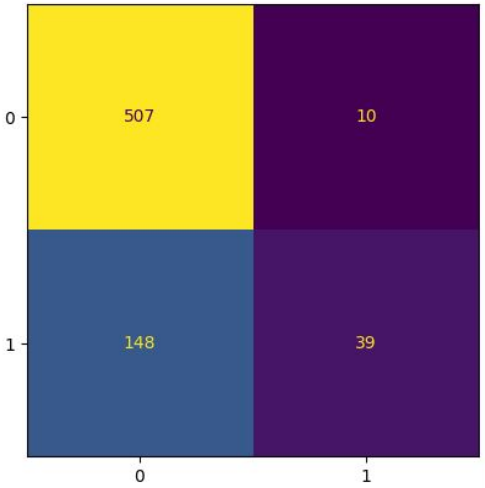
```
0%|          | 2/704 [04:08<24:15:26, 124.40s/it]
```

```
classifier = LazyClassifierFCA()
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
```

[10]

```
100%|██████████| 704/704 [06:17<00:00,  1.86it/s]
```

Accuracy	0.775568
Precision	0.795918
Recall	0.208556
AUC-ROC	0.594607
F1-score	0.330508
True Positive	39.000000
True Negative	507.000000
False Positive	10.000000
False Negative	148.000000
True Negative Rate (Specificity)	0.980658
Negative Predictive Value	0.774046
False Positive Rate	0.019342
False Discovery Rate	0.204082
dtype: float64	



A 200x improvement in computations speed

Mostly predicts the bigger class

Cannot produce class probabilities

IPS: cache results


```python
def get_num_matches(
    self,
    bin_X_train: numpy.ndarray, num_X_train: numpy.ndarray,
    bin_int: numpy.ndarray, num_int_min: numpy.ndarray, num_int_max: numpy.ndarray
):
    bin_match = (bin_X_train | ~bin_int).all(axis = 1)
    num_match = ((num_int_min <= num_X_train) & (num_X_train <= num_int_max)).all(axis = 1)
    return (bin_match & num_match).sum()

def find_matches_impl(self, bin_int: numpy.ndarray, num_int_min: numpy.ndarray, num_int_max: numpy.ndarray, cache):
    if cache:
        key = tuple([*bin_int, *num_int_min, *num_int_max]).__hash__()
        cached_val: bytes = cache.get(key)
        if cached_val:
            num_positive, num_negative = cached_val.split(b';')
            return int(num_positive), int(num_negative)
    num_positive = self.get_num_matches(self.bin_X_train_pos, self.num_X_train_pos, bin_int, num_int_min, num_int_max)
    num_negative = self.get_num_matches(self.bin_X_train_neg, self.num_X_train_neg, bin_int, num_int_min, num_int_max)
    if cache:
        cache.set(key, f"{num_positive};{num_negative}")
    return num_positive, num_negative

def find_matches(self, sample_bin: numpy.ndarray, sample_num: numpy.ndarray, bin: numpy.ndarray, num: numpy.ndarray, cache):
    num_stacked = numpy.vstack([ sample_num, num ])
    return self.find_matches_impl(sample_bin & bin, num_stacked.min(axis = 0), num_stacked.max(axis = 0), cache)

def get_number_of_classifiers(
    self,
    sample_bin: numpy.ndarray, sample_num: numpy.ndarray,
    bin_X_train: numpy.ndarray, num_X_train: numpy.ndarray,
    is_classifier: typing.Callable[[int, int], bool]
):
    if self.use_cache:
        cache: redis.Redis = redis.Redis(host = 'localhost', port = 6379, db = 0)
    else:
        cache = None
    num_classifiers = 0
    for bin, num in zip(bin_X_train, num_X_train):
        if is_classifier(*self.find_matches(sample_bin, sample_num, bin, num, cache)):
            num_classifiers += 1
    return num_classifiers
```

1 Run from empty cache

```python
classifier = LazyClassifierFCA(
    use_cache = True, flush_cache = True
)
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
```

```
100%|██████████| 704/704 [12:26<00:00,  1.06s/it]
```

2 All results cached

```python
classifier = LazyClassifierFCA(use_cache = True)
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)
```

```
100%|██████████| 704/704 [02:24<00:00,  4.86it/s]
```

IPS: improve quality, part 1


```python
def is_positive_classifier(num_positive, num_negative):
    return (self.pos_coef_for_pos_clas * num_positive > num_negative
            and num_positive >= self.min_pos_for_pos_clas)

def is_negative_classifier(num_positive, num_negative):
    return (self.neg_coef_for_neg_clas * num_negative > num_positive
            and num_negative >= self.min_neg_for_neg_clas)

positive_classifiers *= self.pos_clas_coef
total = negative_classifiers + positive_classifiers
return [ (negative_classifiers / total), (positive_classifiers / total) ]
```

```python
fca = LazyClassifierFCA(
  pos_coef_for_pos_clas=2.75,
  neg_coef_for_neg_clas=1/2.75
)
fca.fit(X_train, y_train)
y_pred = fca.predict(X_test)
```

```
3%|          | 704/704 [06:33<00
```

Accuracy	0.768466
Precision	0.639535
Recall	0.294118
AUC-ROC	0.827134
F1-score	0.402930
True Positive	55.000000
True Negative	486.000000
False Positive	31.000000
False Negative	132.000000
True Negative Rate (Specificity)	0.940039
Negative Predictive Value	0.786408
False Positive Rate	0.059961
False Discovery Rate	0.360465
dtype: float64	

- Minimum supporters from ① for a positive classifier
- Weight of supporters from ① relative to opposers from ⓪ for a positive classifier
- Minimum supporters from ⓪ for a negative classifier
- Weight of supporters from ⓪ relative to opposers from ① for a negative classifier
- Weight of positive classifiers relative to negative classifiers when making a final decision

IPS: improve quality, part 2


```
min_pos_for_pos_clas_values = [0, 5, 10, 25, 50, 100, 250, 500]
pos_coef_for_pos_clas_values = [ 0.1, 0.25, 0.5, 0.75, 1.0, 2.5, 5.0, 7.5, 10.0 ]
min_neg_for_neg_clas_values = [0, 5, 10, 25, 50, 100, 250, 500, 1000, 1500 ]
neg_coef_for_neg_clas_values = [ 0.1, 0.25, 0.5, 0.75, 1.0, 2.5, 5.0, 7.5, 10.0 ]
pos_clas_coef_values = [ 0.5, 1.0, 2.0, 2.5, 3.0, 3.5, 4.0, 5.0 ]

100%|██████████| 51840/51840 [10:57:04<00:00,  1.31it/s]
```

```
min_pos_for_pos_clas_values = [ 3, 4, 5, 6, 7 ]
pos_coef_for_pos_clas_values = [ 2.0, 2.25, 2.5, 2.75, 3.0 ]
min_neg_for_neg_clas_values = [ 7, 10, 13, 15, 17 ]
neg_coef_for_neg_clas_values = [ 0.15, 0.2, 0.25, 0.3, 0.35, 0.4 ]
pos_clas_coef_values = [ 0.5, 0.75, 1.0, 1.5, 2.0 ]

100%|██████████| 3750/3750 [50:17<00:00,  1.24it/s]
```

```
classifier = LazyClassifierFCA(
    min_pos_for_pos_clas = 5, pos_coef_for_pos_clas = 2.75,
    min_neg_for_neg_clas = 10, neg_coef_for_neg_clas = 0.25,
    pos_clas_coef = 1.0
)
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)

100%|██████████| 704/704 [06:25<00:00,  1.82it/s]
```

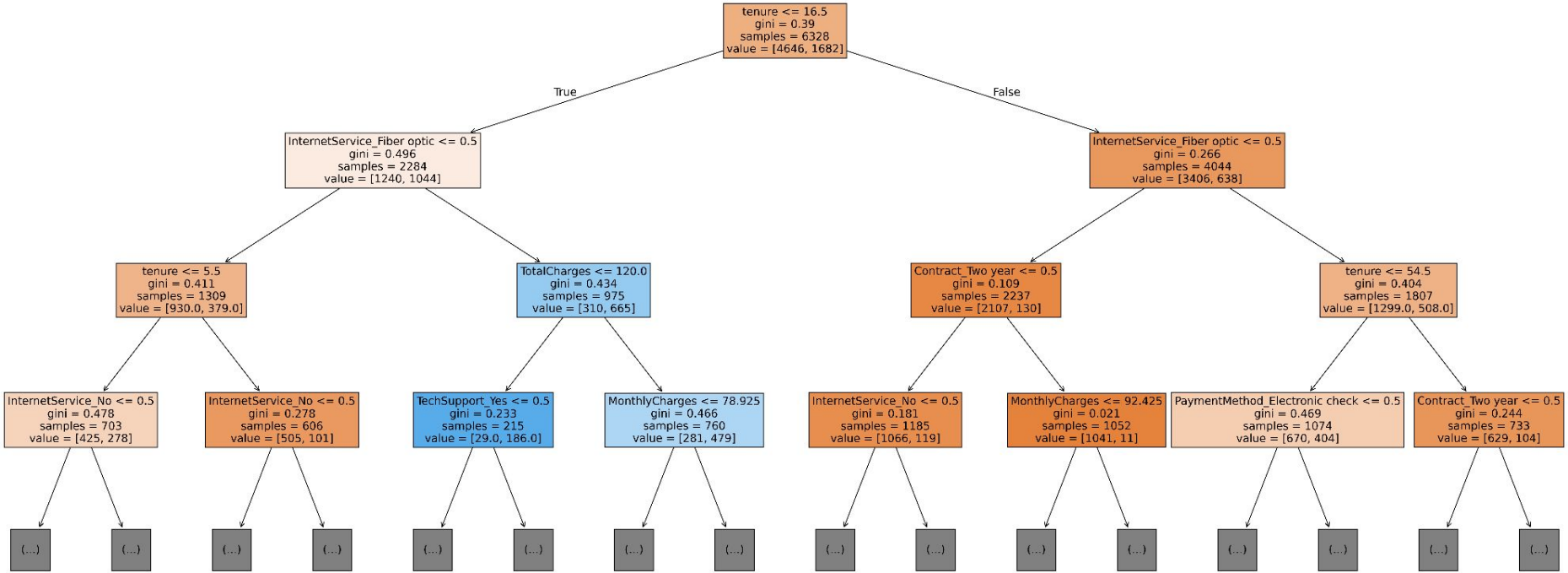
Accuracy	0.755682
Precision	0.526502
Recall	0.796791
AUC-ROC	0.824072
F1-score	0.634043
True Positive	149.000000
True Negative	383.000000
False Positive	134.000000
False Negative	38.000000
True Negative Rate (Specificity)	0.740812
Negative Predictive Value	0.909739
False Positive Rate	0.259188
False Discovery Rate	0.473498

dtype: float64

Comparison of all models

Model	F1 score	Accuracy	AUC – ROC
«Vanilla» IPS	0.33	0.77	0.59
IPS	0.40	0.77	0.83
Best IPS	0.63	0.76	0.82
KNN	0.59	0.8	0.81
Naive Bayes	0.59	0.76	0.81
Logistic Regression	0.59	0.8	0.82
SVM	0.58	0.8	0.81
Decision Tree	0.50	0.79	0.81
Random Forest	0.57	0.79	0.82
Gradient Boosting	0.56	0.79	0.83
XGBoost	0.58	0.8	0.82
CatBoost	0.58	0.8	0.82
MLP	0.55	0.75	0.76

Decision tree interpretability



Interpretation: formal concepts


```cpp
Objects FeaturesPrime(const Features& features)
{
    auto objects = (
        std::views::iota(std::size_t(0), data_.size()) |
        std::views::filter([&](const std::size_t object) {
            return std::ranges::all_of(
                features,
                [&](const std::size_t feature) { return data_[object][feature];}
            );
        })
    );
    Features result;
    for (const std::size_t object : objects) result.push_back(object);
    return result;
}

void Cbo(
    const std::function<bool(const Objects&, const Features&)>& callback,
    const Features& features = {},
    const std::size_t add_from = 0
) {
    const std::set<std::size_t> before_set(features.begin(), features.end());
    const Features closure = FeaturesClosure(features);
    for (const auto& feature : closure)
    {
        if (before_set.contains(feature)) continue;
        if (features.size() != 0 && feature < features[features.size() - 1])
            return; // non canonical
    }
    if (callback(FeaturesPrime(closure), closure))
        return; // Stop if callback tells so
    const std::set<std::size_t> closure_set(closure.begin(), closure.end());
    for (std::size_t i = add_from; i < features_.size(); ++i)
    {
        if (closure_set.contains(i)) continue;
        Features closure_cpy = closure; closure_cpy.push_back(i);
        Cbo(callback, closure_cpy, i + 1);
    }
}
```

Positive concepts:

- SeniorCitizen: 1
- Partner: No
- gender: Female
- OnlineSecurity: No
- PhoneService: Yes
- OnlineBackup: No
- Contract: Month-to-month
- PaperlessBilling: Yes
- PaymentMethod: Electronic check
- SeniorCitizen: 0
- StreamingTV: Yes
- MultipleLines: No
- Dependents: No

Negative concepts:

- InternetService: DSL
- OnlineSecurity: Yes
- Contract: One year
- PaperlessBilling: No
- Dependents: Yes
- Partner: Yes
- TechSupport: Yes
- Contract: Two year
- OnlineBackup: Yes
- SeniorCitizen: 0
- SeniorCitizen: 0
- StreamingMovies: No
- StreamingTV: No
- DeviceProtection: No

Interpretation: utilities


```python
def is_more_general_than(self, other, only_bin: bool = False):
    if self == other: return False
    if (self.bin_int & ~other.bin_int).any(): return False
    if only_bin: return True
    if (self.num_int_min > other.num_int_min).any(): return False
    if (self.num_int_max < other.num_int_max).any(): return False
    return True
```

```python
def minimal_hypotheses(self):
    result: typing.List[Hypothesis] = []
    for key in self.hypotheses:
        for key2 in result:
            if (key.bin_int == key2.bin_int).all():
                break
        else:
            for key2 in self.hypotheses:
                if key2.is_more_general_than(key, only_bin = True):
                    break
            else:
                result.append(key.clone())

    for hypothesis in result:
        for key in self.hypotheses:
            if hypothesis.is_more_general_than(key, only_bin = True):
                hypothesis.num_int_min = numpy.minimum(hypothesis.num_int_min, key.num_int_min)
                hypothesis.num_int_max = numpy.maximum(hypothesis.num_int_max, key.num_int_max)
    return result
```

- Compare the numbers of hypotheses for each class
- Reduce the set of hypotheses to the most general ones preserving the binary part
- Calculate the number of responses of each hypotheses from the reduced set
- Calculate how many times each binary variable appears in the hypotheses
- Compute the distribution of numerical parts of hypotheses

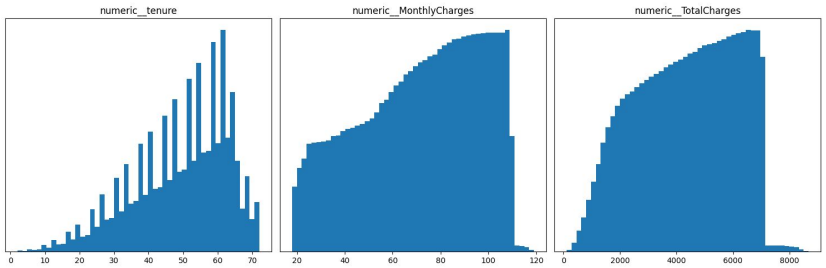
Interpretation: neutral object

Negative classifiers (total 1101):

638x { OnlineBackup: No, PhoneService: Yes, tenure: [2.0; **72.0**], MonthlyCharges: [18.4; 108.95], TotalCharges: [119.5; 7111.3] }

429x { PhoneService: Yes, gender: Female, tenure: [7.0; **72.0**], MonthlyCharges: [18.4; 118.75], TotalCharges: [159.35; 8672.45] }

284x { OnlineSecurity: Yes, PhoneService: Yes, tenure: [2.0; **72.0**], MonthlyCharges: [48.6; 118.75], TotalCharges: [119.5; 8672.45] }

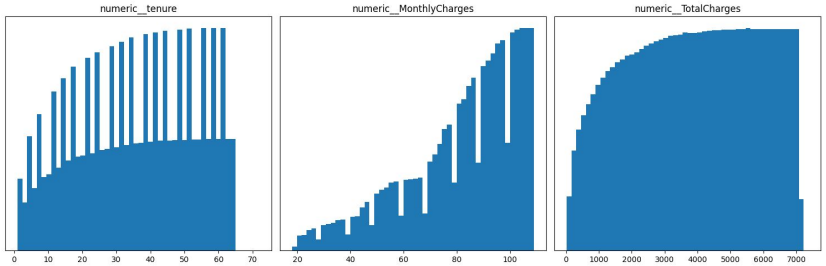


Positive classifiers (total 1199):

1163x { **Contract: Month-to-month**, tenure: [1.0; **64.0**] , MonthlyCharges: [18.95; 108.95], TotalCharges: [19.1; 7111.3] }

562x { Dependents: No, OnlineBackup: No, Partner: No, PhoneService: Yes, tenure: [1.0; **64.0**], MonthlyCharges: [18.95; 108.95], TotalCharges: [19.1; 7111.3] }

296x { PaperlessBilling: Yes, OnlineBackup: No, PhoneService: Yes, gender: Female, tenure: [1.0; **64.0**], MonthlyCharges: [19.1; 108.95], TotalCharges: [19.1; 7111.3] }



	Contract: Month to month	Phone Service: Yes	Dependents: No	Online Backup: No	**Paperless Billing: Yes**	Partner: No	**Internet Service: Fiber optic**	gender: Female	Multiple Lines: Yes	Streaming Movies: Yes	Streaming TV: Yes	**Senior Citizen: 1**	Device Protection: Yes	Payment Method: Bank transfer	**Tech Support: Yes**	**Online Security: Yes**
(+)	**1163**	1080	996	949	**891**	826	**792**	591	449	403	400	**257**	206	96	93	75
(-)	72	948	499	709	348	337	79	502	290	228	224	11	310	285	**388**	**393**

Interpretation: positive object

Negative classifiers (total 86):

77x { TechSupport: No, OnlineSecurity: No, OnlineBackup: No, PhoneService: Yes, DeviceProtection: No, tenure: [**16.0**; 72.0], MonthlyCharges: [18.8; **96.4**], TotalCharges: [279.2; 1874.3] }

6x { StreamingTV: Yes, gender: Female, StreamingMovies: Yes, tenure: [**16.0**; 72.0], MonthlyCharges: [59.55; **96.4**], TotalCharges: [1581.2; 6520.8] }

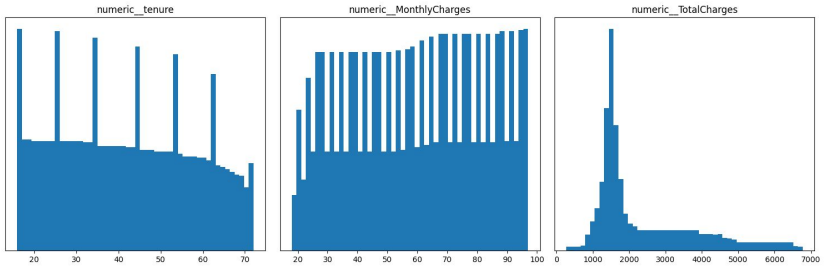
Positive classifiers (total 1243):

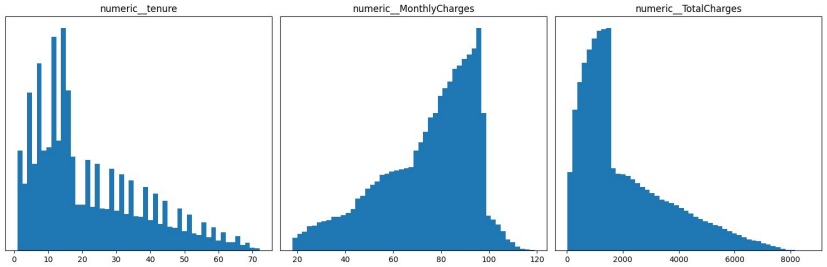
965x { TechSupport: No, **Contract: Month-to-month**, tenure: [**1.0**; 71.0], MonthlyCharges: [18.95; **113.2**], TotalCharges: [19.1; 7548.1] }

808x { DeviceProtection: No, PhoneService: Yes, tenure: [**1.0**; 71.0], MonthlyCharges: [18.95; **111.25**], TotalCharges: [19.1; 7548.1] }

763x { TechSupport: No, Dependents: No, PhoneService: Yes, tenure: [**1.0**; 71.0], MonthlyCharges: [18.95; **113.2**], TotalCharges: [19.1; 7689.95] }

696x { DeviceProtection: No, Contract: Month-to-month, OnlineBackup: No, tenure: [**1.0**; 61.0], MonthlyCharges: [18.95; **103.35**], TotalCharges: [19.1; 5574.75] }





	Phone Service: Yes	Contract: Month to month	Online Security: No	Tech Support: No	Dependents: No	Device Protection: No	Online Backup: No	Paperless Billing: Yes	Partner: No	Internet Service: Fiber optic	Payment Method: Electronic check	gender: Female	Multiple Lines: Yes	Streaming TV: Yes	Streaming Movies: Yes	Senior Citizen: 1
(+)	1113	1086	1051	1042	1014	906	901	891	805	781	678	576	476	426	425	246
(-)	81	1	78	78	27	78	82	5	16	1	0	41	24	7	6	0

Interpretation: negative object

Negative classifiers (total 3545):

2831x { PhoneService: Yes, **SeniorCitizen: 0**, tenure: [1.0; 72.0], MonthlyCharges: [18.55; 118.75], TotalCharges: [18.8; **8672.45**] }

2252x { OnlineSecurity: No, PhoneService: Yes, tenure: [1.0; 72.0], MonthlyCharges: [18.55; 114.1], TotalCharges: [18.8; **8041.65**] }

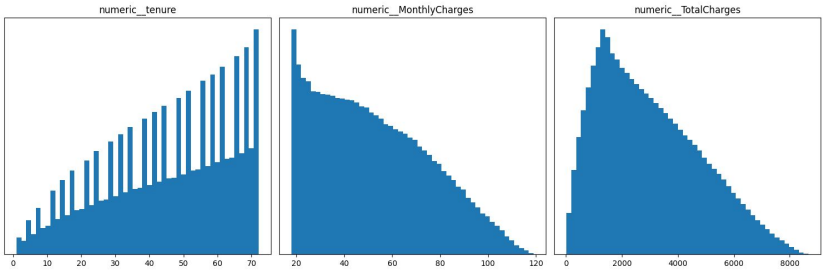
1584x { Partner: Yes, PhoneService:Yes, tenure: [1.0; 72.0], MonthlyCharges: [18.8; 118.75], TotalCharges: [18.8; **8672.45**] }

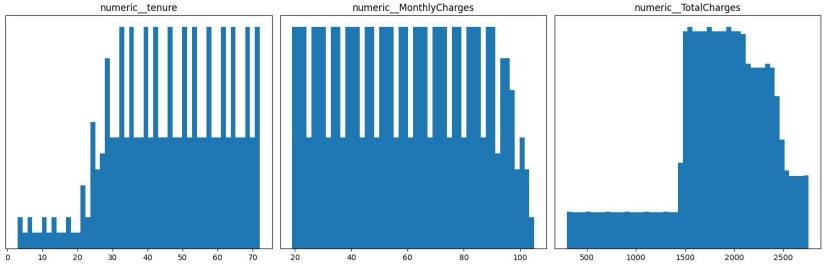
1480x { TechSupport: No, PhoneService: Yes, DeviceProtection: No, StreamingTV: No, tenure: [1.0; 72.0], MonthlyCharges: [18.55; 96.85], TotalCharges: [18.8; **6921.7**] }

1339x { MultipleLines: No, **SeniorCitizen: 0**, StreamingMovies: No, tenure: [1.0; 72.0], MonthlyCharges: [18.55; 95.65], TotalCharges: [18.8; **5817.0**] }

Positive classifiers (total 7):

5x { TechSupport: No, OnlineSecurity: No, PhoneService: Yes, tenure: [3.0; 72.0], MonthlyCharges: [19.55; 105.0], TotalCharges: [294.45; **2755.35**] }





	Phone Service: Yes	Senior Citizen: 0	Online Security: No	Tech Support: No	Device Protection: No	Streaming TV: No	Online Backup: No	Streaming Movies: No	Multiple Lines: No	Partner: Yes	gender: Female	Paperless Billing: No	Dependents: Yes	Internet Service: No	Contract: Two year	Payment Method: Bank transfer
(+)	7	4	6	6	2	0	2	0	6	4	4	0	2	0	0	0
(-)	3284	3057	2413	2385	2300	2235	2233	2207	1814	1677	1656	1537	1049	942	927	701